Smart Linux Service Restart Manager

Faculty Demonstration Guide — terminal commands first, then what to show and say

Important: This guide follows the files and behavior shown in your terminal logs. Your webapp has a short, self-crashing run; the monitor learns a restart budget from `restart_history.csv`. The exact dependency-wait output depends on the monitor's implementation.

Before presenting

Open two Mac Terminal windows. In each, enter the VM. Keep one for the monitor and one for controlling services.

```
multipass start oslab
multipass shell oslab
```

In the second Mac Terminal:

```
multipass shell oslab
```

Confirm the files:

```
ls
```

Expected files include monitor.py, database.sh, webapp.sh, crashy.sh, dashboard.py / dashboard.html, and restart_history.csv.

Demo 1 — Show the problem (plain systemd)

Terminal 2: run these in order. The wait allows the intentionally short-lived webapp to exit.

```
sudo systemctl start webapp.service
sleep 12
systemctl status webapp.service --no-pager
sudo journalctl -u webapp.service -n 10 --no-pager
```

Show: Active: failed, status=1/FAILURE, and journal lines showing it started and crashed.

Say: “This service is designed to fail. systemd detects the failure and records it, but in this demonstration it remains failed without my manager’s recovery logic.”

Demo 2 — Manager attempts recovery

Terminal 2: start the dependency and webapp. (The webapp will crash again after its configured delay.)

```
sudo systemctl start database.service
sudo systemctl start webapp.service
```

Terminal 1: launch the monitor and leave it running.

```
python3 /home/ubuntu/monitor.py
```

Show: a line such as `[action] webapp.service failed — dependencies OK. Restart 1 of 2.` Then show the subsequent active checks. Your prior run showed two restart attempts followed by escalation.

Say: “The monitor sees the webapp failure, checks the database dependency, and attempts a restart when the dependency is healthy.”

Demo 3 — Database down / dependency wait

Keep the monitor running in Terminal 1. Use Terminal 2 to stop the database:

```
sudo systemctl stop database.service
```

Wait for the webapp to fail or for the monitor to check it. Watch Terminal 1 for a dependency-related wait message, for example:

```
[wait] webapp.service failed, but its dependency database.service is down
```

Say only if that output actually appears: “The database is down, so the monitor recognizes the dependency is unavailable and waits rather than restarting the webapp immediately.”

Important: Your earlier monitor run entered HOLD after the webapp exhausted its budget. If it is already held, you may not see the dependency-wait branch. Do not claim Demo 3 succeeded unless the monitor prints the dependency-related wait output.

Demo 4 — Restart budget and escalation

Your captured run already demonstrated this. Show these lines if they appear in the current run:

```
[ESCALATE] webapp.service failed 2 times in 60s (budget 2). Holding.
[held] webapp.service in HOLD — needs admin.
```

Say: “The manager limits repeated restart attempts. After the learned budget is exhausted, it escalates and places the service in HOLD instead of restarting indefinitely.”

Stop the monitor

When you have finished the live demonstration, press Ctrl+C in Terminal 1. This stops the Python monitor; it does not delete your project files.

```
Ctrl+C
```

Restore services after the demo

Run in Terminal 2. Start the database, then check service states:

```
sudo systemctl start database.service
systemctl status database.service --no-pager
systemctl status webapp.service --no-pager
```

The webapp is intentionally crash-prone, so it may show failed again after its delay. That is expected for this test script.

Optional — show dashboard

Inspect the dashboard script before launching it; the correct command/port is not confirmed by the logs provided.

```
head -80 /home/ubuntu/dashboard.py
```

Use the launch instructions defined in that file. Do not assume a port or command.

One-minute closing summary

“This project monitors Linux services through Python and systemd. It checks dependencies before attempting recovery, tracks restart history to determine a restart budget, and escalates repeated failures into HOLD. The demonstrations show failure detection, controlled restart attempts, dependency-aware waiting, and escalation.”

Presentation checklist

- ■ VM running; two terminal windows connected
- ■ Demo 1 failure and journal visible
- ■ Demo 2 dependency-OK restart output visible
- ■ Demo 3 dependency-wait output actually observed (if not, state limitation honestly)
- ■ Escalation/HOLD output visible
- ■ Closing summary ready